

FISA DE LUCRU. PRINCIPII DE ELABORARE A UNUI PROGRAM

Disciplina: Informatica | **Clasa:** a IX-a | **Limbaj utilizat:** Python / Pseudocod

Continuturi vizate	Gandirea computationala. Etapele elaborarii unui program (analiza, proiectare, implementare, testare, depanare). Date de intrare/iesire/manevra. Proiectare modulara. Tipuri de erori (sintaxa, logica, executare).
Activitati vizate	Identificarea datelor unei probleme, descompunerea in module a unei aplicatii, evaluarea erorilor, compararea modurilor de reprezentare a algoritmilor.
Obiective	Elevul recunoaste etapele de dezvoltare a unui program, descompune o problema complexa in subprobleme (module) si clasifica erorile dintr-un cod dat.
Durata recomandata	50 minute

1. Retinem ideea principala

Gandirea computationala reprezinta procesul de formulare a unei probleme si de exprimare a solutiei acesteia in asa fel incat un calculator sa o poata executa. Pentru a crea un program corect, urmam intotdeauna **etapele de elaborare**:

1. **Analiza problemei:** Intelegerea cerintei si identificarea datelor:
 - **Date de intrare:** Ce primim? (ex: doua numere).
 - **Date de iesire:** Ce trebuie sa afisam? (ex: suma lor).
 - **Date de manevra:** Variabile intermediare folosite pentru calcule temporare.
2. **Proiectarea algoritmului:** Alegerea metodei de rezolvare si reprezentarea ei (prin scheme logice, blocuri grafice sau pseudocod). Aici aplicam **proiectarea modulara**: spargerea problemei mari in subprobleme mai mici, independente, numite module (sau subprograme).
3. **Implementarea:** Traducerea algoritmului intr-un limbaj de programare de nivel inalt (ex: Python, C++). Codul sursa este apoi tradus pentru calculator cu ajutorul unui **compiler** sau **interpretor**.
4. **Testarea si Depanarea (Debugging):** Rularea programului cu diferite seturi de date pentru a gasi si corecta erorile.

Categoriile de erori intr-un program:

- **Erori de sintaxa:** Reguli gramaticale incalcate (ex: lipsa unor paranteze sau a doua puncte : in Python). Programul nu porneste.
- **Erori de executare (Runtime):** Apar in timpul rularii programului si il opresc brusc (ex: impartirea la zero, citirea unui fisier inexistent).

- **Erori de logica:** Programul ruleaza perfect, dar produce un rezultat gresit din cauza unei formule sau a unui rationament incorect.

2. Exemple rezolvate

A. Analiza si Modularizarea: Aplicatie pentru gestionarea cheltuielilor

Presupunem ca vrem sa cream o aplicatie care gestioneaza bugetul lunar al unui elev.

- **Analiza datelor:**
 - *Intrare:* Bani de buzunar primiti, cost bilet film, cost mancare.
 - *Iesire:* Mesajul "Te incadrez in buget" sau "Mai ai nevoie de bani", plus soldul ramas.
 - *Manevra:* Variabila care calculeaza suma totala a cheltuielilor.
- **Proiectarea modulara:** In loc sa scriem tot codul la un loc (cod compact, greu de citit), impartim aplicatia in 3 module (functii):
 1. citire_date() - Prelucraza sumele introduse de utilizator.
 2. calcul_sold() - Scade cheltuielile din venituri.
 3. afisare_raport() - Valideaza rezultatul si afiseaza mesajul corespunzator.
- *Avantajul:* Daca apare o eroare la calculul matematic, stim exact ca trebuie sa verificam doar modulul 2, fara sa ne atingem de restul programului.

B. Identificarea tipurilor de erori

Sa privim aceste 3 secvente de cod in Python care incearca sa calculeze media a doua numere, a = 10 si b = 20:

1. `media = (a + b) / 2` (Dar am scris `print(Media)`) -> **Eroare de executare:** Media cu M mare nu este definita.
2. `media = a + b / 2` -> **Eroare de logica:** Din cauza lipsei parantezelor, matematica e gresita (va calcula $10 + 10 = 20$, in loc de 15). Programul afiseaza un rezultat, dar e fals.
3. `if a > b print(a)` -> **Eroare de sintaxa:** Lipsesc cele doua puncte : dupa conditia if. Calculatorul refuza sa citeasca linia.

3. Exerciții diversificate

A. Completeaza spatiile libere

1. Datele initiale pe care un algoritm le primeste pentru a le prelucra se numesc date de _____.
2. Etapa in care traducem algoritmul proiectat pe foaie in cod Python se numeste _____.
3. O aplicatie de marime mare devine mai usor de inteles, impartit in echipa si corectat daca folosim principiul proiectarii _____.
4. Daca un program ruleaza pana la capat, dar calculeaza aria unui patrat folosind formula $latura + latura$ in loc de $latura * latura$, ne confruntam cu o eroare de _____.

B. Adevarat sau fals

1. Un compilator/interpretor are rolul de a traduce codul scris de om (limbaj de nivel inalt) intr-un limbaj de nivel scazut pe care il intelege procesorul. (A / F)
2. Testarea unui program trebuie facuta doar cu date valide, pentru a ne asigura ca functioneaza corect in conditii ideale. (A / F)
3. Datele de manevra sunt afisate obligatoriu pe ecran la finalul rularii programului. (A / F)
4. O solutie modulara este intotdeauna preferabila unei solutii compacte (tip "spaghetti code") cand lucram la proiecte complexe. (A / F)

C. Exerciții de lucru

1. Analiza Datelor: Magazinul de vopsea

O problema ne cere să aflăm câte cutii de vopsea sunt necesare pentru a vopsi peretii unei camere dreptunghiulare (fără tavan și podea). Știm lungimea, lățimea și înălțimea camerei. O cutie de vopsea acoperă 5 metri pătrați.

- Identifica: a) Datele de intrare; b) Cel puțin o dată de manevră necesară; c) Datele de ieșire.

2. Proiectare Modulară: Bancomatul (ATM)

Construiești un meniu pentru un ATM. Descompune această problemă în 4 subprobleme (module) clare. Denumeste fiecare modul și explică în o propoziție ce acțiune face (ex: validarea pin-ului, retragerea etc.).

3. Clasificarea Erorilor

Asociază fiecărei situații de mai jos tipul corect de eroare (Sintaxă, Logică, Executare):

- a) Ai uitat să închizi ghilimelele unui șir de caractere la afișare. (**) b) *Incerți să extragi elementul de pe poziția 10 dintr-o listă care are doar 3 elemente, iar programul crapa instant.* ()**
- c) Ai scris un joc de ghicit numere, dar condiția ta spune dacă `numar_ales < numar_secret`: `print("Numarul tau e prea mare")`. Jocul merge, dar induce jucătorul în eroare. (_____)

Barem orientativ / indicații

- **A:** 1. intrare; 2. implementare; 3. module; 4. logică.
- **B:** 1-A; 2-F (Testarea trebuie să includă și cazuri limită/invalidă pentru a vedea cum reacționează programul la greșeli); 3-F (Datele de manevră sunt invizibile pentru utilizatorul final, folosesc doar pentru memoria intermediară a programului); 4-A.
- **C1:** *Intrare:* Lungime, Lățime, Înălțime (ale camerei). *Manevră:* Aria peretilor (calculată ca perimetru * înălțime). *Ieșire:* Numărul de cutii (Aria împărțită la 5, rotunjit în sus).

- **C2:** Exemple de module: 1. citire_si_validare_card() (verifica PIN-ul); 2. interogare_sold() (afiseaza banii disponibili); 3. retragere_numerar() (scade suma ceruta din sold daca este posibila operatiunea); 4. eliberare_chitanta() (tipareste pe hartie sumarul tranzactiei).
- **C3:** a) Sintaxa; b) Executare (Runtime - "Index Out of Bounds"); c) Logica.